

VİZE PROJE RAPORU

GitHub Linki: <https://github.com/SerraOZSK/244311007-Vize-Odevi>

VERİLERİN ELDE EDİLMESİ

Kullanılan veriler 22.04.2025 tarihinde Dry Bean [Dataset]. (2020). UCI Machine Learning Repository. <https://doi.org/10.24432/C50S4B>. referanslı çalışmada bulunan Dry Bean Dataset excel tablosundan alınmıştır. Projede yapılan tüm işlemler için kullanılan kütüphaneler ve modüller Fig. 1’de gösterilmiştir.

```
#Kullanılan kütüphaneler ve modüller
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, GridSearchCV, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from xgboost import XGBClassifier
from scipy import stats
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import label_binarize
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import roc_curve, auc
```

Fig. 1: Projede kullanılan tüm kütüphane ve modüller

BÖLÜM 1: VERİ ÖN İŞLEME (Preprocessing)

Missing value eklenmesi istenen sütunlar aşağıdaki figürde gösterilmiştir. 5% missing value için “EquivDiameter” ve “roundness” sütunları, 35% missing value için ise “Compactness” sütunu seçilmiştir.

```
#Dataset yüklemek için
df = pd.read_excel("Dry_Bean_Dataset.xlsx")

#Veri setinin ilk birkaç satırını görüntülemek için (emin olmak için önemli)
print("Initial data snapshot:")
print(df.head())

#Veride halihazırda eksik veri var mı görmek için
print(df.isnull().sum())

#İki rastgele sütuna 5% missing value eklemek için
for col in ['EquivDiameter', 'roundness']:
    df.loc[df.sample(frac=0.05, random_state=42).index, col] = np.nan
#Bu sayede real-life data corruption simüle edilmiş oluyor

#Başka bir sütuna 35% missing value eklemek için
df.loc[df.sample(frac=0.35, random_state=7).index, 'Compactness'] = np.nan
#Burada da daha büyük bir veri eksikliği çalışılıyor, veriler yanlışlıkla silinebilir veya belli faktörler yüzünden elde edilemeyebilir

#Veri setini cvs dosyası olarak dışa aktarmak için
df.to_csv("Dry_Bean_Missing_Example.csv", index=False)

#Veri setinin ne kadar broken olduğunu görmek için
print("\nMissing values per column:")
print(df.isnull().sum())
```

Fig. 2: Veri setinde seçilen iki sütuna 5%, bir sütuna ise 35% missing value eklenmesi

Eklenen missing value'lar .cvs dosyasına dönüştürülmüştür ve github'a eklenmiştir. Bu rapora eklenen Fig. 3 ve Fig. 4'te bulunan sonuçlar print fonksiyonu kullanılarak elde edilmiştir. Fig. 3, missing value eklenmeden önceki ilk birkaç veriyi (Fig.3A) ve işlem öncesi veri setindeki missing value sayısını (Fig.3B) göstermektedir. Missing value ekleme işlemi sonrasında değerler Fig. 4'te gözüktüğü şekilde yazdırılmıştır.

Initial data snapshot:							Area	0
	Area	Perimeter	MajorAxisLength	MinorAxisLength	AspectRation	\	Perimeter	0
0	28395	610.291	208.178117	173.888747	1.197191		MajorAxisLength	0
1	28734	638.018	200.524796	182.734419	1.097356		MinorAxisLength	0
2	29380	624.110	212.826130	175.931143	1.209713		AspectRation	0
3	30008	645.884	210.557999	182.516516	1.153638		Eccentricity	0
4	30140	620.134	201.847882	190.279279	1.060798		ConvexArea	0
	Eccentricity	ConvexArea	EquivDiameter	Extent	Solidity	roundness	EquivDiameter	0
0	0.549812	28715	190.141097	0.763923	0.988856	0.958027	Extent	0
1	0.411785	29172	191.272750	0.783968	0.984986	0.887034	Solidity	0
2	0.562727	29690	193.410904	0.778113	0.989559	0.947849	roundness	0
3	0.498616	30724	195.467062	0.782681	0.976696	0.903936	Compactness	0
4	0.333680	30417	195.896503	0.773098	0.990893	0.984877	ShapeFactor1	0
	Compactness	ShapeFactor1	ShapeFactor2	ShapeFactor3	ShapeFactor4	Class	ShapeFactor2	0
0	0.913358	0.007332	0.003147	0.834222	0.998724	SEKER	ShapeFactor3	0
1	0.953861	0.006979	0.003564	0.909851	0.998430	SEKER	ShapeFactor4	0
2	0.908774	0.007244	0.003048	0.825871	0.999066	SEKER	Class	0
3	0.928329	0.007017	0.003215	0.861794	0.994199	SEKER	dtype: int64	B
4	0.970516	0.006697	0.003665	0.941900	0.999166	SEKER		

Fig. 3: A) Elde edilen veri setindeki ilk 5 değer. B) İşlem yapılmadan önce veri setinde var olan missing value sayıları

Missing values per column:	
Area	0
Perimeter	0
MajorAxisLength	0
MinorAxisLength	0
AspectRatio	0
Eccentricity	0
ConvexArea	0
EquivDiameter	681
Extent	0
Solidity	0
roundness	681
Compactness	4764
ShapeFactor1	0
ShapeFactor2	0
ShapeFactor3	0
ShapeFactor4	0
Class	0
dtype:	int64

Fig. 4: EquivDiameter ve roundness sütunlarına eklenen 5% ve Compactness sütununa eklenen 35% missing value sayıları

Veri setinin eksik veri barındırması modelin çalışmasını kötü etkileyecek ve stabilitesini azaltacağından dolayı 5% eksik değerler ortalama ile doldurulmuş, ancak %35 eksik değer içeren sütun göz ardı edilemeyecek kadar eksik veri barındırdığından dolayı silinmesine karar verilmiştir. Bu aşama için kullanılan kod Fig. 5'te gösterilmiş ve print fonksiyonu kullanılarak var olan eksik değer sayısını gösteren çıktı ise Fig. 6'da bulunmaktadır.

```
#5% missing value olan sütunları ortalama ile doldurmak gerekiyor
df['EquivDiameter'] = df['EquivDiameter'].fillna(df['EquivDiameter'].mean())
df['roundness'] = df['roundness'].fillna(df['roundness'].mean())

#35% missing sütun bu dakikadan itibaren bizim için ölmüştür:
df = df.drop(columns=['Compactness'])
#Çalışma için kritik değilse ve çok fazla eksik veri barındırıyorsa, onu tutmak çok riskli olacaktır

#Veri setinin son halini kontrol etmek için
print("\nRemaining missing values:")
print(df.isnull().sum())
```

Fig. 5: 5% eksik veriler ortalama değerle değiştirmek ve 35% eksik değerlerin bulunduğu sütunu silmek için yapılan işlemler

```
Remaining missing values:  
Area          0  
Perimeter     0  
MajorAxisLength 0  
MinorAxisLength 0  
AspectRation  0  
Eccentricity   0  
ConvexArea     0  
EquivDiameter  0  
Extent         0  
Solidity       0  
roundness      0  
ShapeFactor1   0  
ShapeFactor2   0  
ShapeFactor3   0  
ShapeFactor4   0  
Class          0  
dtype: int64
```

Fig. 6: Yapılan işlem sonrası var olan eksik değer sayısını gösteren çıktı.

Fig. 7, Z-score uygulamak için gereken işlemleri göstermektedir. Aykırı değerleri belirlemek için Z-score kullanılmıştır. Z-score normal bir dağılım gösteren veriler arasındaki aykırı değer tespiti için kullanılır ve farklı birimlerdeki verileri karşılaştırmak için elverişlidir. Bu nedenlerden dolayı Z-score seçilmiştir. Aşağıdaki girdide bulunan Z-score < 3 şartının konması güven aralığını 99.7%'ye çıkartmak için eklenmiştir.

```
#Outlier'ları silmek için Z-score  
z_scores = np.abs(stats.zscore(df.select_dtypes(include=[np.number])))  
df = df[(z_scores < 3).all(axis=1)]  
#Z-score 3'ten büyük olanları siliyoruz (güven aralığı %99.7)
```

Fig. 7: Aykırı değer tespiti için kullanılan Z-score girdisi

Veri setindeki veri aralıkları farklı olduğunda modelin öğrenmesi ve işlem yapması zorlaşacağı için StandardScaler kullanılarak ölçeklerin aynı aralıkta olması sağlanmıştır. Label Encoder kullanarak “Class” sütunu numerik hale getirilmiştir. Bu işlemler için Fig. 8’de bulunan girdi kullanılmıştır. Numerik olmayan tek sütun “Class” olduğu için başka bir işlem yapılmamıştır.

```
#StandardScaler ile verileri aynı ölçeğe getirmek için
scaler = StandardScaler() #bir veri 1-10000 aralığındayken diğeri 0-1 arasındaysa modelin öğrenmesi zorlaşır
num_cols = df.select_dtypes(include=[np.number]).columns
df[num_cols] = scaler.fit_transform(df[num_cols])

#LabelEncoder ile Class sütunlarını numerik hale dönüştürmek için
le = LabelEncoder()
df['Class'] = le.fit_transform(df['Class'])

#Veri setinin son halini kontrol etmek için
print("\nFinal preprocessed data:")
print(df.head())

#Veri setindeki 0 değeri -> tam olarak ortalamaya eşit
#          (-) değeri -> ortalamanın altında
#          (+) değeri -> ortalamanın üstünde
#Classlar fasulye genotiplerine göre encode edildi
```

Fig. 8: StandardScaler ve LabelEncoder işlemlerini içeren girdi

Aşağıda StandardScaler kullanılarak aynı ölçeğe getirilmiş veriler bulunmaktadır. Bu verilerde negative ve pozitif değerler gözlemlenmektedir. Negatif değer, verinin ortalamasının altında olduğunu, pozitif değerler ise verinin ortalamasının üstünde olduğunu ifade etmektedir. Çıktıda tam olarak 0 değeri gözlemlenmese de, 0 olması verinin ortalamaya eşit olduğunu bildirmektedir. Class sütununun kategorik verilerine numerik bir kimlik atanmıştır.

```
Final preprocessed data:
   Area  Perimeter  MajorAxisLength  MinorAxisLength  AspectRatio  \
0 -1.179696 -1.340630      -1.507916      -0.721257      -1.582858  \
2 -1.120449 -1.253114      -1.437459      -0.652072      -1.530552  \
3 -1.082676 -1.115221      -1.471840      -0.428995      -1.764796  \
5 -1.066376 -1.184611      -1.441484      -0.463084      -1.691989  \
6 -1.054466 -0.962286      -1.464380      -0.377420      -1.793493  \

   Eccentricity  ConvexArea  EquivDiameter  Extent  Solidity  roundness  \
0    -2.290172   -1.177045    -1.346331  0.282084  0.353266   1.480018  \
2    -2.142456   -1.119431    -1.264990  0.588445  0.550526   1.294350  \
3    -2.875724   -1.058331    -1.213840  0.687063 -3.059968   0.493257  \
5    -2.626564   -1.065658    -1.191933  0.536097  0.536781   1.221422  \
6    -2.980240   -1.043794    -1.175988  0.249247 -0.986914  -0.434504  \

   ShapeFactor1  ShapeFactor2  ShapeFactor3  ShapeFactor4  Class
0    0.659101    2.465712    1.965041    0.960919    5
2    0.568288    2.288776    1.877880    1.065063    5
3    0.332758    2.585272    2.252791   -0.416023    5
5    0.336217    2.475468    2.163824    1.116685    5
6    0.237555    2.634065    2.350806    1.059768    5
```

Fig. 9: Aynı ölçeğe indirgenmiş verilerin ve numerikleştirilmiş Class sütunu çıktısı

BÖLÜM 2: ÖZELLİK SEÇİMİ VE BOYUT İNDİRGEME

```
#preprocessed veriyi kaydetmek için
X = df.drop('Class', axis=1)
y = df['class']
X_ham = X.copy()
y_ham = y.copy()

#PCA uygulamak için
pca = PCA()
X_pca_full = pca.fit_transform(X)

#Average variance explained by each component hesaplamak için
explained_variance = pca.explained_variance_ratio_
average_variance = np.mean(explained_variance)

# PCA bileşenlerini seçmek için
selected_components = np.where(explained_variance > average_variance)[0]
print(f"Seçilen PCA bileşenleri: {selected_components}")

#Seçilen bileşenleri azaltmak için
pca_selected = PCA(n_components=len(selected_components))
X_pca = pca_selected.fit_transform(X)

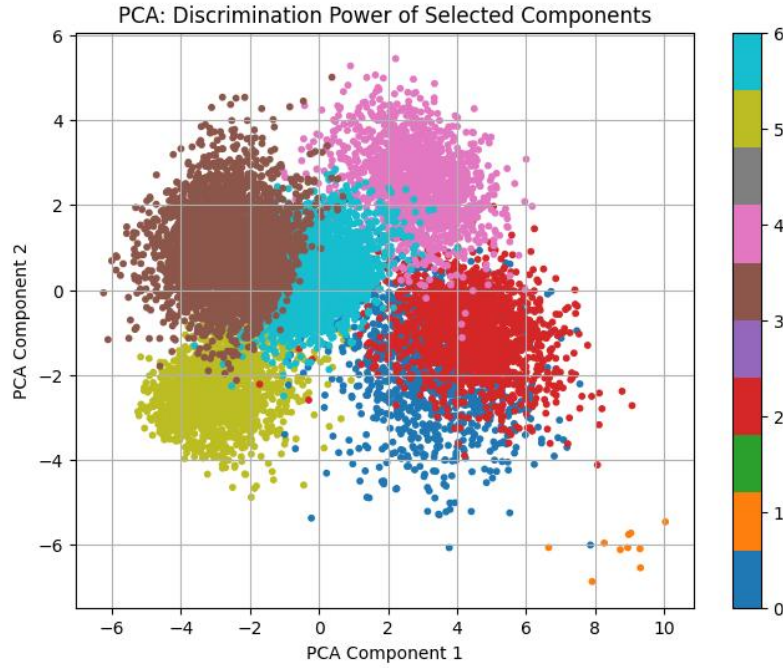
#PCA verilerini kaydetmek için
X_pca_data = X_pca
y_pca_data = y.copy()

#PCA verilerini 2D olarak görselleştirmek için
plt.figure(figsize=(8,6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='tab10', s=10)
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.title('PCA: Discrimination Power of Selected Components')
plt.colorbar()
plt.grid(True)
plt.show()
```

Fig. 10: Ham (Preprocessed) veriler üzerinde PCA uygulaması

Fig. 10, PCA kullanılarak boyut indirgeme işleminin girdisini içermektedir. Seçilecek özvektör sayısı açıklanabilen değişiklik yüzdesinin (percentage of explained variances) ortalamasından büyük olacak şekilde şartlandırılmıştır. Bu sayede veri setinde bilgi kaybı minimize edilirken aynı zamanda gereksiz bileşenler modelden arındırılmış olmaktadır. Bu girdi kullanılarak iki boyutlu bir grafik oluşturulmuştur.

Grafik 1: En iyi iki özneliğin discrimination power'ını gösteren PCA grafiği



Yukarıdaki PCA grafiğinde farklı renkler, farklı sınıfların verilerini temsil etmektedir. İdeal bir PCA grafiğinde renk kümelerinin birbirinden olabildiğince ayrık olması, üst üste gelmemeleri gerekmekte ve bu sayede iki bileşenin ayırım gücünün oldukça yüksek olduğu anlaşılmaktadır. Grafik 1’de bulunan PCA grafiği ise bu çalışmada kullanılan veri setindeki varyansı açıklayan en etkili ilk iki ana bileşenin ayırım gücünü görselleştirmiştir. Grafiğin sağında bulunan renk skalası, veri noktalarının hangi sınıfa ait olduğunu işaret etmektedir. Mavi ve kırmızı renkle gösterilen verilerin üst üste geldiği, Turkuaz, kahverengi ve yeşil renkle gösterilen verilerin ise bir kısmının çakıştığı görülmektedir. Aynı zamanda turuncu renkle gösterilen sınıfa ait veriler tam anlaşılamamaktadır. Her ne kadar üst üste gelen veriler olsa da kümelerin genel olarak birbirlerinden ayrık ve belirgin oldukları görülmektedir. Bu yukarıdaki PCA grafiğini iyi bir grafik yapmaktadır. Ancak PCA, sınıflar arası ayırım yapamadığından yeterli olmamakta ve daha ayrıntılı bir ayırım için LDA uygulanmasına ihtiyaç duyulmaktadır.


```
#3 bileşenli LDA uygulamak için
lda = LDA(n_components=3)
x_lda = lda.fit_transform(X,y)

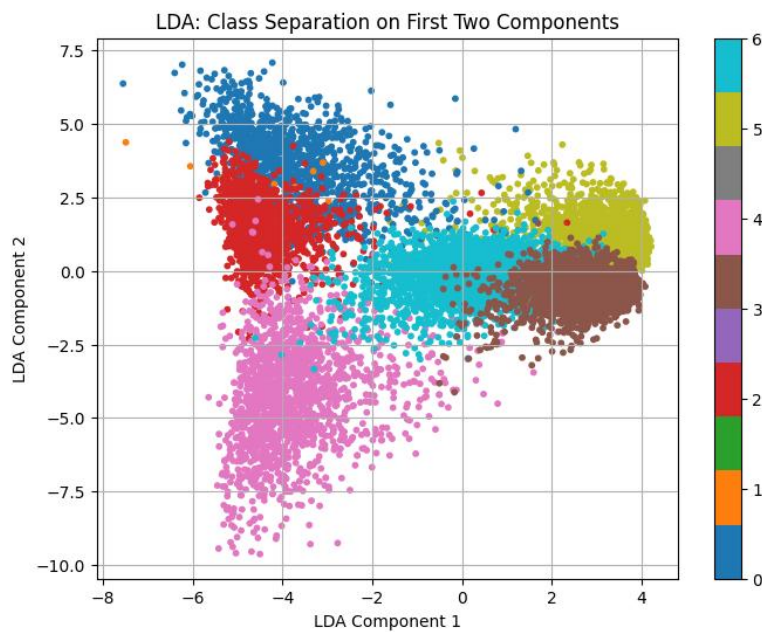
#LDA verilerini kaydetmek için
x_lda_data = x_lda
y_lda_data = y.copy()

#İlk iki LDA bileşenini görselleştirmek için
plt.figure(figsize=(8,6))
plt.scatter(x_lda[:, 0], x_lda[:, 1], c=y, cmap='tab10', s=10)
plt.xlabel('LDA Component 1')
plt.ylabel('LDA Component 2')
plt.title('LDA: Class Separation on First Two Components')
plt.colorbar()
plt.grid(True)
plt.show()
```

Fig. 11: LDA uygulamak için kullanılan girdi

PCA uygulaması öncelikli olarak boyut indirgemek için kullanıldığından genel varyansı maksimize eder ancak sınıflar arası ayrımı göstermekte yetersiz kalır. LDA, PCA'nin yetersiz kaldığı bu aşamada kullanılır. LDA'nin sınıfları belirgin bir şekilde ayırabiliyor olması PCA'ne kıyasla daha iyi bir ayrım gücü sunar. Fig. 11'de LDA uygulaması için oluşturulan girdi bulunmaktadır. Üç boyutlu bir LDA uygulamak için seçilecek özvektör sayısı 3 olarak şartlandırılmıştır. İlk iki LDA bileşeni ile analiz sonuçlarını iki boyutlu bir grafik olarak sunması istenmiştir.

Grafik 2: Sınıflar arası ayrımı gösteren LDA grafiği



Yukarıdaki grafikte sınıflar arası ayrımı en iyi ilk iki bileşene göre görselleştirmiş bir LDA uygulaması gözlemlenmektedir. Genel olarak bakıldığında, LDA grafiği daha belirgin ayrımlar sunmaktadır. PCA grafiğinde daha ayırık olarak gözlemlenen veri noktalarının (örneğin mavi ve kırmızı renkle kodlanmış veri sınıfları) LDA grafiğinde kendi aralarında daha fazla kümelenmiş oldukları gözlemlenmektedir. Ancak kırmızı ve turuncu kodlu sınıflar diğer sınıflarla çakışan veri noktalarına sahiptir. Pembe veri sınıfı biraz dağınık gözükse de yeterli biçimde belirgindir ve dağınıklığı içindeki varyansın yüksek olmasından kaynaklı olabilir. Kahverengi ve yeşil kodlu sınıflar ise belirgin kümelenme göstermektedir. Grafikte gözlemlenen belirgin ayrım, işleme tabi tutulan özelliklerin sınıflandırma açısından anlamlı olduğunu göstermektedir.

BÖLÜM 3: MODELLEME ve DEĞERLENDİRME

Fig. 12, 14, ve 16'da gösterilen aşamalarda ilk olarak modelin performansının daha tarafsız bir şekilde değerlendirilmesi için iç içe geçmiş çapraz doğrulama (Nested Cross Validation) işlemi uygulanmıştır. Outer loop, yani dış döngü, veriyi farklı test ve eğitim setlerine bölen 5 katmanlı Stratified Cross-Validation olarak yapılandırılmıştır. Bu işlem, veri setinin beş parçaya ayrılmasını ve bunlardan birinin test seti diğer dördünün ise eğitim ve doğrulama setleri olarak kullanılmasını sağlar. Dış döngülerdeki eğitim setlerine 3 katmanlı Stratified Cross-Validation iç döngü olarak atanmış ve GridSearchCV kullanılarak hiperparametre optimizasyonu yapılmıştır. Hiperparametreler arasında iç döngüde en iyi performansı gösterenler dış test seti üzerinde kullanılmıştır. Bu sayede hiperparametre ayarlanırken test setlerindeki verilere bilgi sızdırılmasının önüne geçilmiştir. Böylelikle bu aşamada yapılan performans değerlendirmesi, tüm dış döngülerden elde edilen test sonuçlarının ortalaması ve standart deviation sonuçları hesaplanarak modelin genellenebilirliği hakkında bilgi vermektedir. Birbirlerinden farklı olarak fig. 12 ham (unprocessed) veriyi, fig. 13 PCA uygulanmış veriyi ve fig. 14 LDA uygulanmış veriyi işlenen veri seti olarak almıştır. PCA ve LDA veri setleri NumPy dizisi olduğu için .iloc[] olarak okunamaz. Bu yüzden Pandas DataFrame'e çevrilmiştir.

```
#Ham veri için dataset
X_use = X_ham
y_use = y_ham

#Outer CV (5-fold), random_state her loopta değişiyor
outer_cv = StratifiedKFold(n_splits=5, shuffle=True)
#Inner CV (5-fold), random_state sabit
inner_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

#Hyperparameter belirlemek için
param_grid = {
    'C' : [0.01, 0.1, 1, 10, 100],
    'penalty': ['l2'],
    'solver': ['lbfgs']
}
outer_scores = []

#Outer loop için
fold_idx = 1
for train_idx, test_idx in outer_cv.split(X_use, y_use):
    print(f"Starting Outer Fold {fold_idx}...")
    X_train, X_test = X_use.iloc[train_idx], X_use.iloc[test_idx]
    y_train, y_test = y_use.iloc[train_idx], y_use.iloc[test_idx]

    #Inner Loop için
    model = LogisticRegression(max_iter=1000)
    grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=inner_cv, scoring='accuracy')
    grid_search.fit(X_train, y_train)
    best_model = grid_search.best_estimator_

    #En iyi modeli outer test seti üzerinde değerlendirmek için
    test_score = best_model.score(X_test, y_test)
    outer_scores.append(test_score)

    print(f"Fold {fold_idx} Accuracy: {test_score:.4f}")
    fold_idx += 1

#Sonuçları yazdırmak için
print("\nOuter Nested CV Accuracy:")
print(f"Mean Accuracy: {np.mean(outer_scores):.4f}")
print(f"Standard Deviation: {np.std(outer_scores):.4f}")
```

Fig. 12: Ham veri kullanılarak Nested Cross-Validation yapılandırmak için oluşturulan girdi

```
#PCA ve LDA verileri Numpy dizisinde olduğu için .iloc[] olarak okunmayacak
#Pandas DataFrame'e dönüştürmek için
X_pca_data = pd.DataFrame(X_pca_data)
y_pca_data = pd.Series(y_pca_data)

#PCA verileri için aynı işlemler
X_use = X_pca_data
y_use = y_pca_data

#Outer CV ve Inner CV için
outer_cv = StratifiedKFold(n_splits=5, shuffle=True)
inner_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

#Hyperparameter belirlemek için
param_grid = {
    'C' : [0.01, 0.1, 1, 10, 100],
    'penalty': ['l2'],
    'solver': ['lbfgs']
}
outer_scores = []

#Outer Loop için
fold_idx = 1
for train_idx, test_idx in outer_cv.split(X_use, y_use):
    print(f"Starting Outer Fold {fold_idx}...")
    X_train, X_test = X_use.iloc[train_idx], X_use.iloc[test_idx]
    y_train, y_test = y_use.iloc[train_idx], y_use.iloc[test_idx]

    #Inner Loop için
    model = LogisticRegression(max_iter=1000)
    grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=inner_cv, scoring='accuracy')
    grid_search.fit(X_train, y_train)
    best_model = grid_search.best_estimator_

    #En iyi modeli outer test seti üzerinde değerlendirmek için
    test_score = best_model.score(X_test, y_test)
    outer_scores.append(test_score)

    print(f"Fold {fold_idx} Accuracy: {test_score:.4f}")
    fold_idx += 1

#Sonuçları yazdırmak için
print("\nOuter Nested CV Accuracy:")
print(f"Mean Accuracy: {np.mean(outer_scores):.4f}")
print(f"Standard Deviation: {np.std(outer_scores):.4f}")
```

✓ 14.2s

Fig. 13: PCA uygulanmış veri kullanılarak Nested Cross-Validation yapılandırmak için oluşturulan girdi

```
x_lda_data = pd.DataFrame(X_lda_data)
y_lda_data = pd.Series(y_lda_data)

#LDA verileri için aynı işlemler
X_use = X_lda_data
y_use = y_lda_data

#Outer CV ve Inner CV için
outer_cv = StratifiedKFold(n_splits=5, shuffle=True)
inner_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

#Hyperparameter belirlemek için
param_grid = {
    'C' : [0.01, 0.1, 1, 10, 100],
    'penalty': ['l2'],
    'solver': ['lbfgs']
}
outer_scores = []

#Outer loop için
fold_idx = 1
for train_idx, test_idx in outer_cv.split(X_use, y_use):
    print(f"Starting Outer Fold {fold_idx}...")
    X_train, X_test = X_use.iloc[train_idx], X_use.iloc[test_idx]
    y_train, y_test = y_use.iloc[train_idx], y_use.iloc[test_idx]

    #Inner loop için
    model = LogisticRegression(max_iter=1000)
    grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=inner_cv, scoring='accuracy')
    grid_search.fit(X_train, y_train)
    best_model = grid_search.best_estimator_

    #En iyi modeli outer test seti üzerinde değerlendirmek için
    test_score = best_model.score(X_test, y_test)
    outer_scores.append(test_score)

    print(f"Fold {fold_idx} Accuracy: {test_score:.4f}")
    fold_idx += 1

#Sonuçları yazdırmak için
print("\nOuter Nested CV Accuracy:")
print(f"Mean Accuracy: {np.mean(outer_scores):.4f}")
print(f"Standard Deviation: {np.std(outer_scores):.4f}")
```

Fig. 14: LDA uygulanmış veri kullanılarak Nested Cross-Validation yapılandırmak için oluşturulan girdi

Tablo 1. Kullanılan veri temsilleri için Nested CV işlemi sonuçları

Veri Tipi	Outer Fold 1	Outer Fold 2	Outer Fold 3	Outer Fold 4	Outer Fold 5	Mean Accuracy	Standard Deviation
Ham Veri	0.9235	0.9119	0.9303	0.9255	0.9251	0.9233	0.0061
PCA Verisi	0.8694	0.8734	0.8943	0.8846	0.871	0.8786	0.0095
LDA Verisi	0.9075	0.9111	0.9071	0.9054	0.9131	0.9088	0.0028

Yukarıdaki tablo farklı veri temsillerini (ham, PCA ve LDA uygulanmış veriler) kullanarak yapılan Nested CV operasyonunun sonuçlarını barındırmaktadır. Her satırda 5 dış katmandaki accuracy değerlerini, bu değerlerin ortalamasını ve standart sapmasını göstermektedir. Ortalama accuracy ve standart sapma değerleri karşılaştırıldığında ham yani unprocessed veriyi kullanan model en yüksek doğruluk oranına (0.9233) ve oldukça düşük bir standart sapma değerine (0.0061) sahip. Modelin bu veriyi kullanarak sınıflandırma işlemini oldukça iyi bir şekilde yerine getirdiği ve kararlı bir performansa sahip olduğu söylenebilir. PCA uygulanmış veriyi kullanan model, diğer verileri kullanan modellere kıyasla en düşük ortalama accuracy (0.8786) ve en büyük standart sapma değerine (0.0095) sahip. Değerlerdeki ciddi farklılık, boyut indirgeme işlemi sırasında yaşanan bilgi kaybından dolayı olabilir. LDA uygulanmış veriyi kullanan model ise iyi bir denge göstermektedir. Bu modelin hem ortalama accuracy değerinin (0.9088) ham veriye yakın doğruluk oranına hem de en düşük standart sapma değerine (0.0028) sahip olması ile en stabil performansa sahip olduğu görülmektedir.

```
#Model ve grid tanımlamak için
model_and_grid = {
    "Logistic Regression": {
        "model": LogisticRegression(max_iter=1000),
        "params": {
            'C': [0.01, 0.1, 1, 10],
            'penalty': ['l2'],
            'solver': ['lbfgs']
        }
    },
    "Decision Tree": {
        "model": DecisionTreeClassifier(),
        "params": {
            'max_depth': [3, 5, 10, None],
            'min_samples_split': [2, 5, 10]
        }
    },
    "Random Forest": {
        "model": RandomForestClassifier(),
        "params": {
            'n_estimators': [50, 100],
            'max_depth': [5, 10, None]
        }
    },
    "XGBoost": {
        "model": XGBClassifier(eval_metric='mlogloss'),
        "params": {
            'n_estimators': [50, 100],
            'max_depth': [3, 5],
            'learning_rate': [0.01, 0.1]
        }
    },
    "Naive Bayes": {
        "model": GaussianNB(),
        "params": {}
    }
}
```

1

```
#Tekrar kullanılabilir nested CV fonksiyonu tanımlamak için

def run_nested_cv_all_models(X, y, models_and_grids):
    results = {}
    outer_cv = StratifiedKFold(n_splits=5, shuffle=True)
    inner_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

    for name, config in models_and_grids.items():
        print(f"\nRunning model: {name}")
        outer_scores = []

        fold_idx = 1
        for train_idx, test_idx in outer_cv.split(X, y):
            X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
            y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

            if config["params"]:
                grid = GridSearchCV(estimator=config["model"], param_grid=config["params"], cv=inner_cv, scoring='accuracy')
                grid.fit(X_train, y_train)
                best_model = grid.best_estimator_
            else: #Naive Bayes için çünkü ona parametre belirlenmedi
                best_model = config["model"]
                best_model.fit(X_train, y_train)

            score = best_model.score(X_test, y_test)
            outer_scores.append(score)
            print(f"Fold {fold_idx} Accuracy: {score:.4f}")
            fold_idx += 1

        mean_acc = np.mean(outer_scores)
        std_acc = np.std(outer_scores)
        results[name] = {"Mean Accuracy": mean_acc, "Standard Deviation": std_acc}

    return pd.DataFrame(results).T

X_ham = pd.DataFrame(X_ham)
y_ham = pd.Series(y_ham)
X_pca_data = pd.DataFrame(X_pca_data)
y_pca_data = pd.Series(y_pca_data)
X_lda_data = pd.DataFrame(X_lda_data)
y_lda_data = pd.Series(y_lda_data)

results_ham = run_nested_cv_all_models(X_ham, y_ham, model_and_grids)
results_pca = run_nested_cv_all_models(X_pca_data, y_pca_data, model_and_grids)
results_lda = run_nested_cv_all_models(X_lda_data, y_lda_data, model_and_grids)

#Sonuçları tablolar halinde yazdırmak için
print("\nResults for Ham Data:")
print(results_ham)

print("\nResults for PCA Data:")
print(results_pca)

print("\nResults for LDA Data:")
```

Fig. 15: Cross-Validation’da her bir veri temsilini kullanacak sınıflandırıcıların girdisi

Fig. 15’te gösterilen aşamada toplam 5 adet sınıflandırma algoritmasını üç veri temsili üzerinde uygulayarak tüm modellerin aynı veri dağılımı ve Cross-Validation yapısı altında değerlendirilmesi amaçlanmaktadır. Bu sayede model performansları algoritmalar ve veri temsilleriyle sınırlandırılır ve rastgele veri bölünmesinden kaynaklı sapmaların önüne geçilmiş olur. Kullanılan sınıflandırıcılar sırasıyla Logistic Regression, Decision Tree, Random Forest, XGBoost, Naïve Bayes’tir. Fig. 15’te de görüldüğü gibi bu süreçte inner loop (iç döngü) hiperparametre optimizasyonu için ve outer loop (dış döngü) ise modelin genel başarısını bağımsız test verisi üzerinde değerlendirmek için şartlanmıştır. Bu sayede hiperparametre bilgilerinin test veri setine sızması engellenmiştir. Modeller, tüm veri temsilleri üzerinde 5 katmanlı outer cross validation ile test edilmiş ve mean accuracy ve standard deviation çıktıları karşılaştırılmıştır. Bu sayede hem model seçimi ve hem de hangi veri temsili stratejisinin modeli

daha iyi desteklediği karşılaştırmalı olarak belirlenmiştir. Fig. 15’te bulunan girdi uzun olduğu için 3 parçaya ayrılmış ve 1, 2, 3 olarak işaretlenmiş bir şekilde bu rapora eklenmiştir.

Tablo 2. Ham veriler kullanılarak sınıflandırılan modelin Outer Fold sonuçları

Ham Veri	Outer Fold 1	Outer Fold 2	Outer Fold 3	Outer Fold 4	Outer Fold 5
Logistic Regression	0.9259	0.9183	0.9239	0.9199	0.9267
Desicion Tree	0.9035	0.8959	0.8987	0.9111	0.9038
Random Forest	0.9211	0.9231	0.9215	0.9147	0.9135
XGBoost	0.9211	0.9235	0.9335	0.9163	0.9235
Naive Bayes	0.8991	0.8999	0.8911	0.8874	0.8918

Tablo 3. PCA uygulanmış veriler kullanılarak sınıflandırılan modelin Outer Fold sonuçları

PCA Veri	Outer Fold 1	Outer Fold 2	Outer Fold 3	Outer Fold 4	Outer Fold 5
Logistic Regression	0.8807	0.8795	0.8722	0.8834	0.8758
Desicion Tree	0.8602	0.8546	0.8582	0.8518	0.8602
Random Forest	0.8899	0.8771	0.8815	0.8714	0.873
XGBoost	0.8823	0.8851	0.8734	0.8746	0.8838
Naive Bayes	0.8686	0.8755	0.8638	0.8682	0.8654

Tablo 4. LDA uygulanmış veriler kullanılarak sınıflandırılan modelin Outer Fold sonuçları

LDA Veri	Outer Fold 1	Outer Fold 2	Outer Fold 3	Outer Fold 4	Outer Fold 5
Logistic Regression	0.9135	0.9079	0.9115	0.9075	0.9018
Desicion Tree	0.8883	0.8975	0.8963	0.8918	0.8918
Random Forest	0.9043	0.8987	0.9211	0.9127	0.9071
XGBoost	0.9035	0.9083	0.9067	0.9159	0.9127
Naive Bayes	0.9051	0.8911	0.8899	0.8898	0.8974

Tablo 5. Tüm Veri Temsili ve Sınıflandırıcıların Ortalama Accuracy ve Standart Sapmalarının Kıyaslanması

	Ham Veri		PCA Veri		LDA Veri	
	Mean Accuracy	Standard Deviation	Mean Accuracy	Standard Deviation	Mean Accuracy	Standard Deviation
Logistic Regression	0.922935	0.003308	0.878315	0.003897	0.908435	0.003992
Desicion Tree	0.902588	0.005186	0.857005	0.00332	0.893134	0.003342
Random Forest	0.918769	0.003919	0.878554	0.006646	0.908756	0.007645
XGBoost	0.923575	0.005629	0.879836	0.004842	0.909398	0.004382
Naive Bayes	0.893855	0.004835	0.868301	0.003992	0.894657	0.005927

Tablolar 2, 3 ve 4, her bir sınıflandırma algoritmasının sırasıyla ham veri, PCA uygulanmış veri ve LDA uygulanmış veri temsilleri üzerinde nested cross-validation uygulayarak değerlendirilmiş 5 outer fold accuracy skorlarını içermektedir. Bu skorlar doğrultusunda her bir veri temsili ve algoritma için ortalama accuracy ve standart sapma değerleri hesaplanmış ve çıktılar Tablo 5'te kaydedilmiştir. Tablo 5'teki değerleri kıyaslayacak olursak, ham veri kullanan modeller arasında en yüksek accuracy oranına (92.35%) sahip olan XGBoost modelidir. Bu modelin ardından Logistic Regression modeli (92.29%) gelmekte ve onu da Random Forest (91.88%) takip etmektedir. Bu durum ham verinin herhangi bir boyut indirgeme işlemi uygulanmadan da doğruluk seviyesinin yüksek olduğunu göstermektedir. Genel olarak PCA verilerini kullanan tüm modellerde düşük skorlar gözlenmektedir. Doğruluk oranı en yüksek olan modeller sırasıyla XGBoost (87.98%), Random Forest (87.86%) ve Logistic Regression (87.83%) olsa da ham veri kullanan modellere ait verilere kıyasla accuracy oranları azdır. Bu durum, PCA uygulamasının bilgi kaybına yol açtığını ve model performansının bundan olumsuz etkilenebileceğini göstermektedir. Bu değerlerin yanında LDA uygulanan verileri kullanan modeller PCA verilerine göre daha yüksektir. En iyi değer yine XGBoost modeline (90.94%) ait olup sırasıyla Random Forest (90.88%) ve Logistic Regression (90.84%) modelleri tarafından takip edilmektedir. Tüm veri temsilleri için XGBoost istikrarlı bir şekilde yüksek accuracy ortalamasına sahiptir. Standart sapma değerleri ise benzer düşüklükte olup 0.003-0.007 arasında değişmektedir. Bu durum, modellerin farklı katmanlarda verdiği sonuçların tutarlı olup overfitting riskinin azaltıldığına işaret etmektedir.

```
def run_nested_cv_all_metrics(X, y, models_and_grids):
    results = {}
    outer_cv = StratifiedKFold(n_splits=5, shuffle=True)
    inner_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

    for name, config in models_and_grids.items():
        print(f"\nRunning model: {name}")

        acc_list, prec_list, rec_list, f1_list = [], [], [], []

        for train_idx, test_idx in outer_cv.split(X, y):
            X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
            y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

            if config["params"]:
                grid = GridSearchCV(estimator=config["model"], param_grid=config["params"], cv=inner_cv, scoring='accuracy')
                grid.fit(X_train, y_train)
                best_model = grid.best_estimator_
            else:
                best_model = config["model"]
                best_model.fit(X_train, y_train)

            y_pred = best_model.predict(X_test)

            acc_list.append(accuracy_score(y_test, y_pred))
            prec_list.append(precision_score(y_test, y_pred, average='macro', zero_division=0))
            rec_list.append(recall_score(y_test, y_pred, average='macro'))
            f1_list.append(f1_score(y_test, y_pred, average='macro'))

        results[name] = {
            'Accuracy (mean)': np.mean(acc_list),
            'Accuracy (std)': np.std(acc_list),
            'Precision (mean)': np.mean(prec_list),
            'Precision (std)': np.std(prec_list),
            'Recall (mean)': np.mean(rec_list),
            'Recall (std)': np.std(rec_list),
            'F1 (mean)': np.mean(f1_list),
            'F1 (std)': np.std(f1_list),
        }

    return pd.DataFrame(results).T

# Tüm verisetlerinin doğru formatta olduğundan emin olmak için kontrol tekrarı
X_ham = pd.DataFrame(X_ham)
y_ham = pd.Series(y_ham)

X_pca_data = pd.DataFrame(X_pca_data)
y_pca_data = pd.Series(y_pca_data)

X_lda_data = pd.DataFrame(X_lda_data)
y_lda_data = pd.Series(y_lda_data)

# Ham veri, PCA verisi ve LDA verisi için tüm metrikleri çalıştırmak için
print("\nRunning on raw data:")
results_ham_metrics = run_nested_cv_all_metrics(X_ham, y_ham, model_and_grids)

print("\nRunning on PCA data:")
results_pca_metrics = run_nested_cv_all_metrics(X_pca_data, y_pca_data, model_and_grids)

print("\nRunning on LDA data:")
results_lda_metrics = run_nested_cv_all_metrics(X_lda_data, y_lda_data, model_and_grids)

results_ham_metrics['Data Type'] = 'Ham Veri'
results_pca_metrics['Data Type'] = 'PCA Veri'
results_lda_metrics['Data Type'] = 'LDA Veri'

all_metrics = pd.concat([results_ham_metrics, results_pca_metrics, results_lda_metrics])
all_metrics.reset_index(inplace=True)
all_metrics.rename(columns={'index': 'Model'}, inplace=True)

#Tablo oluşturmak için
print("\nFinal Performance Metrics Summary:")
print(all_metrics)
```

Fig. 16: Modellerin sonuçlarını accuracy, precision, recall, ve F1 score metrikleri ile elde etmek için hazırlanan girdi

Tablo 6. Ham veri, PCA verisi ve LDA verisi kullanan modellerin belirlenen metrikler ile oluşturulan sonuç çıktısı

Veri Temsili	Model	Accuracy (mean)	Accuracy (std)	Precision (mean)	Precision (std)	Recall (mean)	Recall (std)	F1 (mean)	F1 (std)
Ham Veri	Logistic Regression	0.922455	0.002712	0.93799	0.002662	0.922449	0.026043	0.928594	0.017423
	Decision Tree	0.90427	0.003596	0.880122	0.056369	0.850276	0.043447	0.860293	0.045473
	Random Forest	0.920452	0.00575	0.937631	0.005046	0.91948	0.026214	0.926001	0.017286
	XGBoost	0.924138	0.006982	0.941679	0.004631	0.895282	0.034248	0.911217	0.023148
	Naive Bayes	0.894336	0.00556	0.910254	0.005678	0.89543	0.029756	0.89989	0.020027
PCA Veri	Logistic Regression	0.878876	0.00288	0.878278	0.017043	0.869262	0.032055	0.869631	0.02171
	Decision Tree	0.858368	0.00419	0.826828	0.057981	0.821923	0.059051	0.823503	0.058594
	Random Forest	0.879115	0.006201	0.880094	0.017093	0.854903	0.037758	0.860668	0.023282
	XGBoost	0.880477	0.004079	0.881105	0.018015	0.869574	0.030829	0.871063	0.020992
	Naive Bayes	0.869742	0.006538	0.873999	0.024823	0.86118	0.028883	0.860779	0.022338
LDA Veri	Logistic Regression	0.908835	0.00596	0.92634	0.003942	0.909019	0.031074	0.915138	0.021388
	Decision Tree	0.889529	0.004169	0.871892	0.056832	0.862987	0.057718	0.863785	0.053625
	Random Forest	0.909397	0.002925	0.927487	0.003052	0.908589	0.027693	0.915459	0.018219
	XGBoost	0.908435	0.001574	0.88416	0.034757	0.879155	0.034788	0.876697	0.026259
	Naive Bayes	0.894497	0.002274	0.902936	0.018269	0.890394	0.030508	0.891455	0.021231

Fig. 16'nın girdisini içerdiği aşamada Ham veri, PCA uygulanmış veri ve LDA uygulanmış veri kullanan beş farklı modelde kullanmak için dört performans metriği atanmıştır. Bu metrikler sırasıyla Accuracy, Precision, Recall ve F1 skorudur. Sonuçlar, Tablo 2, 3 ve 4'te bulunan 5 katmanlı outer loop değerlendirmelerinin ortalamaları ve standart sapma değerleri olarak Tablo 6'da bulunmaktadır. Tablo 6'daki metrik değerleri sayesinde doğruluklarının yanında, modellerin kararlılıklarını değerlendirmek mümkündür. Ortalama accuracy ve accuracy'ye ait olan değerlerin karşılaştırması Tablo 5'in altında tartışılmıştır. Ham veri kullanan modeller arasında en yüksek precision değerine XGBoost (94.16%), Recall değerine Logistic Regression (92.24%), F1 Skoruna ise yine Logistic Regression (92.86%) sahiptir. PCA uygulanan veriler daha önce karşılaştırılan sonuçlar gibi bu metriklerde de diğer veri temsilleri arasında en düşük değerlere sahiptir. Özellikle, Decision Tree modelinin precision, recall, ve F1 skoru ortalamaları (~82%) oldukça geridedir. LDA uygulanan verileri kullanan modeller ise önceki karşılaştırma sonuçlarıyla benzer başarıda çıktılar vermiş, dengeli olma durumunu korumuştur. Ancak en yüksek değerlere sahip modeller, precision için Random Forest (92.75%), recall için Logistic Regression (90.9%), F1 skoru için Random Forest (91.56%) olmuştur. Genel olarak modellerin accuracy, precision, recall ve F1 skoru performansları farklılık göstermiştir.

```
datasets = [  
    (X_ham, y_ham, "Ham Veri"),  
    (X_pca_data, y_pca_data, "PCA Verisi"),  
    (X_lda_data, y_lda_data, "LDA Verisi")  
]  
  
for model_name, config in model_and_grids.items():  
    for X_data, y_data, data_label in datasets:  
        model = config["model"]  
        params = config["params"]  
  
        # Fix for XGBoost  
        if isinstance(model, XGBClassifier):  
            model.set_params(num_class=len(np.unique(y_data)))  
  
        full_label = f"{model_name} ({data_label})"  
        print(f"\nROC eğrisi çiziliyor: {full_label}")  
        plot_roc_curve_ova_best(X_data, y_data, model=model, param_grid=params, label=full_label)
```

Fig. 17: Modellerne ait ROC Eğrilerinin eldesi için hazırlanan girdi

Fig. 17’de bulunan girdi kullanılarak oluşturulan ROC eğrileri, sınıflandırıcı modelin performansını her sınıf için FP ve TP oranları arasındaki değerlendirmeden yola çıkarak gösterir ve doğruluk seviyesini ölçer. OVA yöntemi ile ayırım güçlerini net bir şekilde analiz edebilmek için her sınıfa ait farklı bir ROC eğrisi oluşturulur. AUC değerinin 1 olması, modelin tüm verileri doğru analiz ettiğine işaret eder.

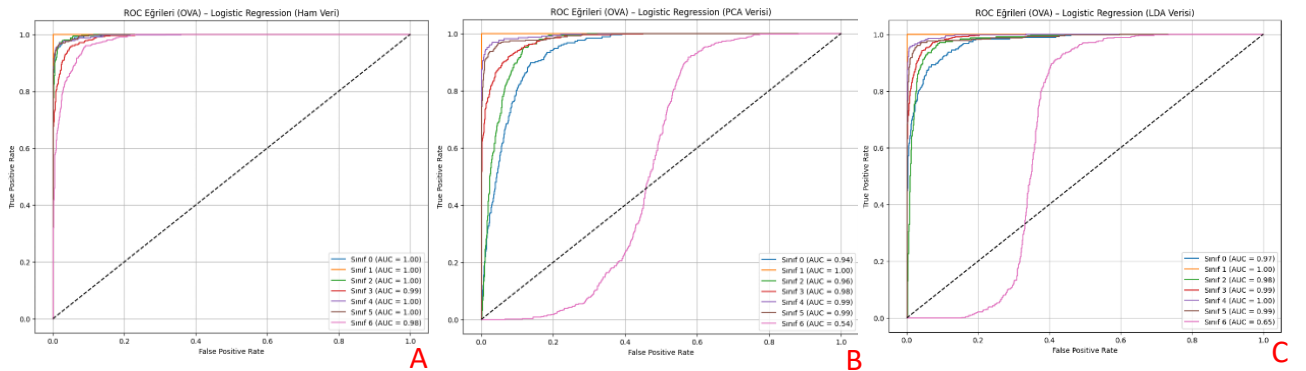


Fig. 18: Logistic Regression modelinin farklı veri temsilleri için oluşturulmuş ROC eğrileri ve AUC skorları. A) Ham veriye ait eğri. B) PCA verisine ait eğri. C) LDA Verisine ait eğri.

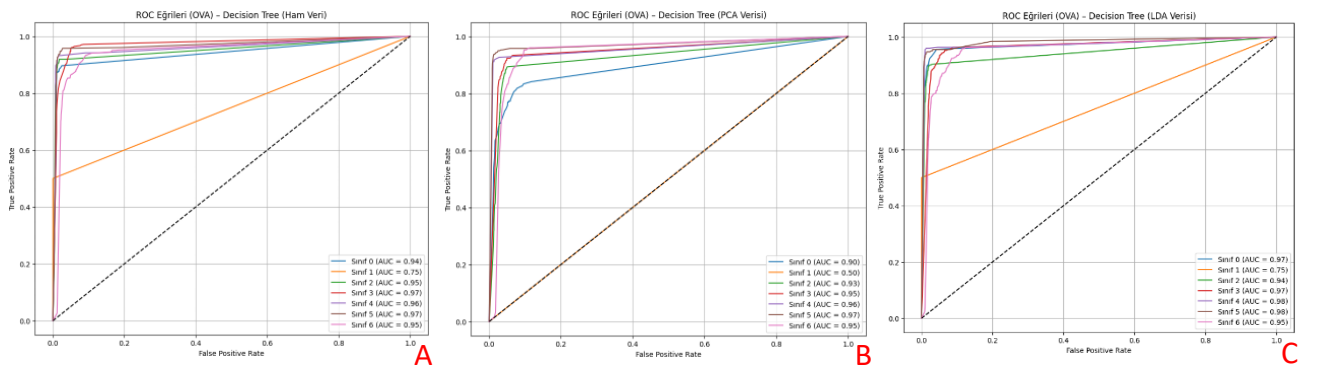


Fig. 19: Decision Tree modelinin farklı veri temsilleri için oluşturulmuş ROC eğrileri ve AUC skorları. A) Ham veriye ait eğri. B) PCA verisine ait eğri. C) LDA Verisine ait eğri.

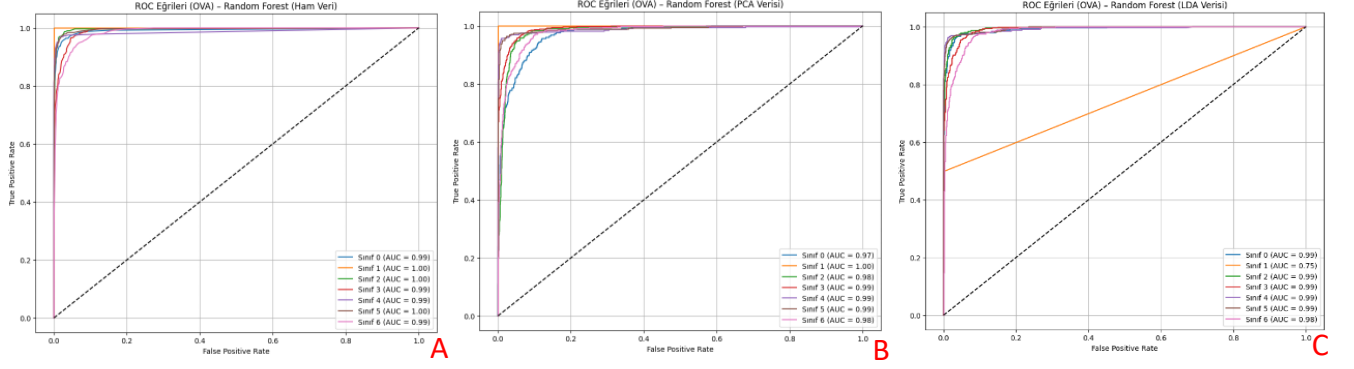


Fig. 20: Random Forest modelinin farklı veri temsilleri için oluşturulmuş ROC eğrileri ve AUC skorları. A) Ham veriye ait eğri. B) PCA verisine ait eğri. C) LDA Verisine ait eğri.

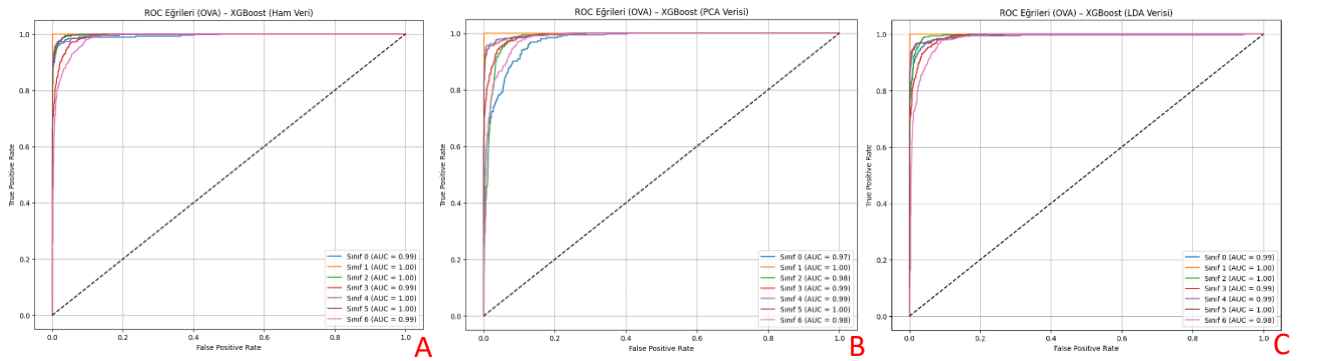


Fig. 21: XGBoost modelinin farklı veri temsilleri için oluşturulmuş ROC eğrileri ve AUC skorları. A) Ham veriye ait eğri. B) PCA verisine ait eğri. C) LDA Verisine ait eğri.

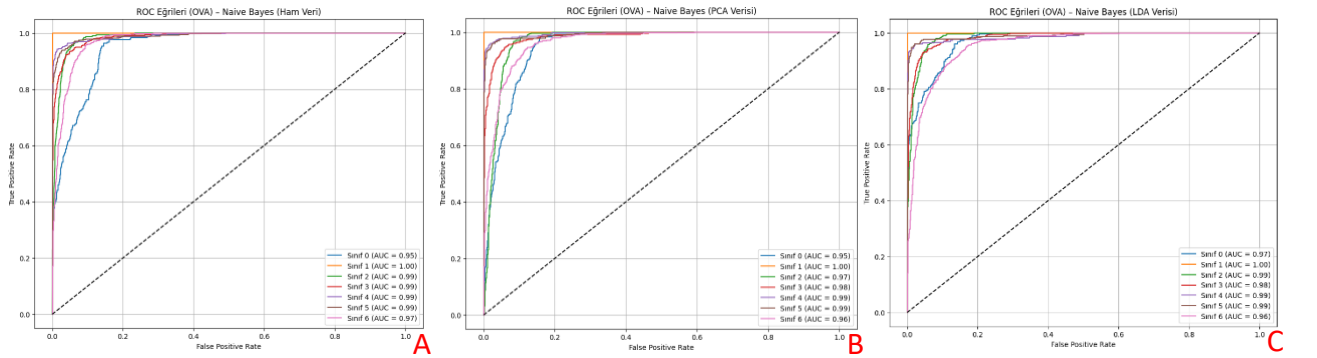


Fig. 22: Naive Bayes modelinin farklı veri temsilleri için oluşturulmuş ROC eğrileri ve AUC skorları. A) Ham veriye ait eğri. B) PCA verisine ait eğri. C) LDA Verisine ait eğri.

Yukarıda proje boyunca oluşturulmuş ve işleyebilir hale getirilmiş modellerin (Linear Regression, Fig 18; Decision Tree, Fig 19; Random Forest, Fig 20; XGBoost, Fig 21; Naive Bayes, Fig 22) farklı veri temsilleri (A)Ham Veri, B) PCA uygulanmış veri, C) LDA uygulanmış veri) üzerindeki performans değerlerini ifade eden ROC eğrileri ve AUC değerleri bulunmaktadır. Fig. 18'in içerdiği Linear Regression eğrilerinde her ne kadar ham veriye ait eğrilerin AUC değerleri çoğunlukla 1.00 olsa da, PCA ve LDA uygulanan verilere ait Fig. 17B ve 17C grafiklerinde Sınıf 6 eğrisinin performansının çok düşüktür. Fig. 19'da bulunan

Decision Tree eğrileri ise hiçbir veri temsili için yeterli performansa sahip olduğunu gösterememiştir. AUC değerleri genellikle 0.93-0.97 (Fig 19A, B, C) arasında olsa da henüz ham veri setinde (Fig 19A) bile sınıf 1 verilerini iyi bir şekilde işleyememiştir. Fig. 20'de bulunan eğrilere sahip Random Forest modeli ham veri (Fig 20A) ve PCA verisi (Fig 20B) performansları oldukça iyi performans sergilemektedir. Ancak, şaşırtıcı bir şekilde, LDA verileri (Fig 20C) üzerinde aynı etkide bir performans gösterememiş, sınıf 1 AUC değeri 0.75'e düşmüştür. Bu durum modelin güvenilirliğini azaltmaktadır. Fig. 21'de bulunan XGBoost eğrileri, tüm ROC eğrileri arasında en kararlı ve yüksek performansı göstermiştir ve tüm AUC değerleri 0.98-1.00 aralığındadır. Son olarak, Naive Bayes modelinin eğrilerini içeren Fig. 22'de, Fig. 18, 19 ve 20'de bulunan ani AUC değeri değişiklikleri gözlemlenmez. Ancak AUC değerlerine bakıldığında performansının XGBoost modeli kadar yüksek olduğu söylenememektedir.

SONUÇ

Bu çalışma boyunca bir veri seti elde edilmiş, veri seti eksik değerler eklenerek, bir nevi, gerçek hayat veri toplama aşaması simüle edilmiş, boyut indirgeme analizleri yapılmış, dış ve iç döngüler kullanılarak bir Nested Cross-Validation yapısı oluşturulmuş, bu doğrultuda beş model oluşturulmuş ve farklı metrikler ile model performansları ölçülmüş, ve son olarak modellerin ROC eğrileri oluşturularak en iyi performansa sahip model ve veri temsili belirlenmesi aşama aşama incelenmiştir.

Nested Cross-Validation ile elde edilen accuracy değerleri ve performans metriklerinin değerlendirmeleri modelin başarısını indike etmektedir. XGBoost modeli ham veri üstünde en yüksek accuracy oranına sahiptir. Bunun yanında, Tablo 5'te verilere bakılarak, PCA sürecindeki boyut indirgeme işleminin bilgi kaybına yol açtığı ve bu durumun modellerin doğruluk oranlarını düşürdüğü düşünülmektedir. Fig. 18B ve C grafiklerinde sınıf 6 verilerinde gözlemlenen değişikliğin PCA işleminden sonra başlaması ve LDA işlemine aktarılmış olması bu durumu destekleyebilir niteliktedir. LDA uygulaması ise PCA uygulamasına kıyasla daha dengeli sonuçlar verse de bazı modellerde sınıf ayrımının zorlaştığı gözlemlenmiştir. Tüm veri temsilleri üzerinde en düşük performansı Decision Tree modeli göstermiştir.

XGBoost modeli, tüm modeller arasındaki en kararlı ve yüksek performansa sahip model olarak öne çıkmıştır. Bu durum Fig 21'de bulunan ROC eğrileri ve AUC değerleri ile desteklenmiştir. En iyi performans sıralamasında bu modeli Random Forest ve Logistic Regression modelleri takip etmektedir. Her ne kadar bazı veri temsilleri ve metrik değerlendirmelerinde daha güçlü sonuçlar sunsalar da skorları istikrar göstermemektedir.

Sonuç olarak, sadece bir model tüm sınıflandırmalarda mükemmel performansı gösteremeyeceğinden, çalışılan veri temsiline göre tercih edilen modelin değişmesi gerekebilmektedir.

REFERANSLAR:

KOKLU, M. and OZKAN, I.A., (2020), "Multiclass Classification of Dry Beans Using Computer Vision and Machine Learning Techniques." Computers and Electronics in Agriculture, 174, 105507. DOI: <https://doi.org/10.1016/j.compag.2020.105507>